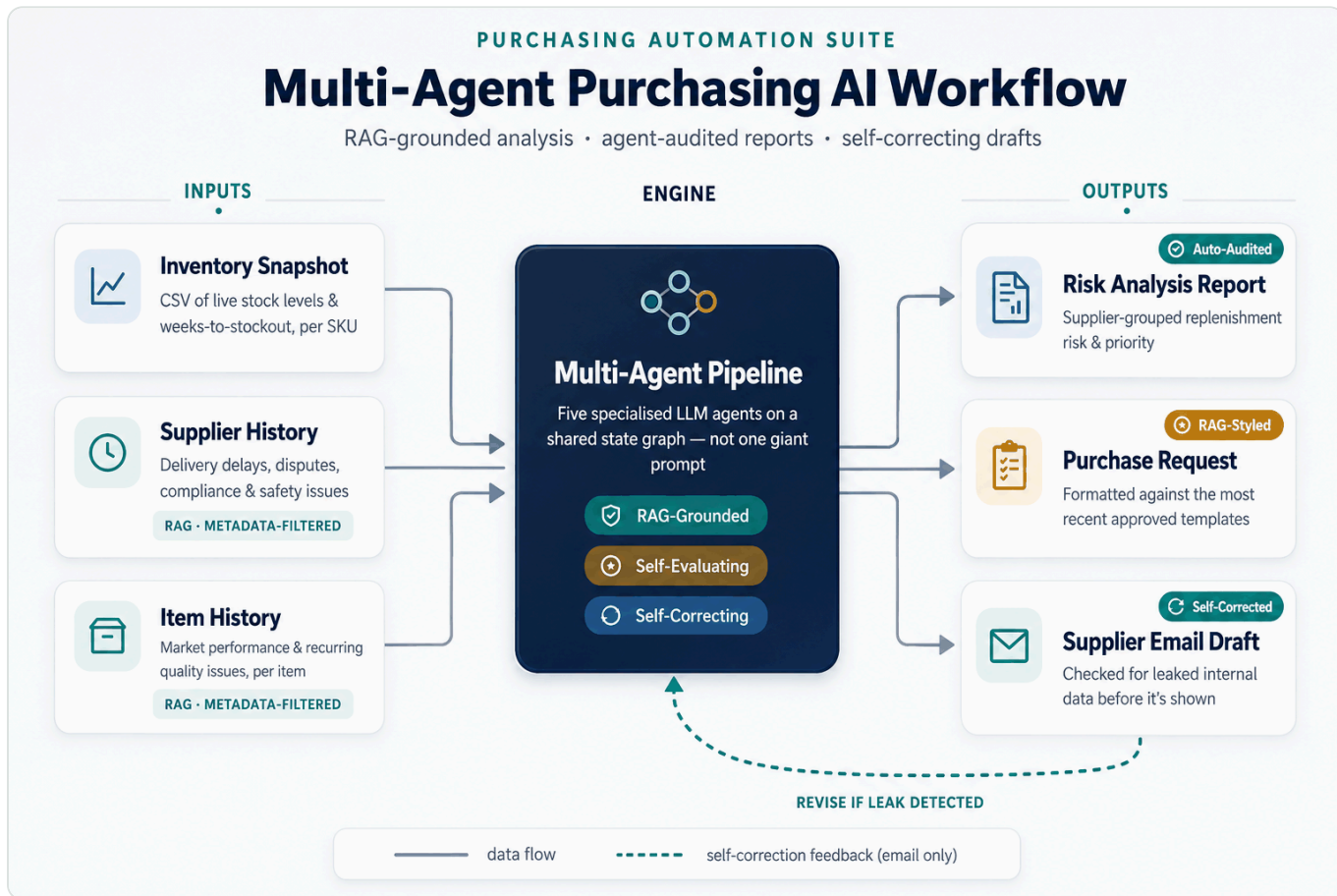


구매 의사결정 지원 멀티 에이전트 AI 시스템

(Multi-agent Purchasing AI Suite)

재고 스냅샷과 공급업체·아이템 히스토리 컨텍스트를 RAG로 함께 분석해, 구매 담당자들이 수작업으로 하던 **재고 리스크 분석·충당 계획 수립부터 분석 보고서·구매 요청서·공급업체 이메일 초안 작성까지**를 대신하는 멀티 에이전트 LLM 파이프라인.

특히 단순 재고 확인에 그치지 않고, 공급업체의 납기 준수·배송 지연 이력, 거래·협상 내역, 통관·규제·안전 이슈부터 해당 품목의 시장 퍼포먼스와 품목별 문제 이력까지 — 여러 문서에 흩어져 있던 관련 맥락을 RAG를 통해 자동으로 끌어와 분석합니다. 생성된 결과물은 **별도 에이전트가 데이터·논리 일관성을 자체 검수해 품질 평가 보고서를 함께 생성하며**, 이메일 초안은 내부 정보 유출 여부를 자동 점검해 문제가 있으면 스스로 재작성하는 자기 교정 루프를 거칩니다.



데모: <https://soominmyung.com/purchasing-automation>

소스: <https://github.com/soominmyung/purchasing-automation>

한눈에 보기

기간	2025년 10월 – 2025년 12월
역할	단독 프로젝트 — 아키텍처 설계, 개발, 배포
분야	생성형 AI 기반 재고 리스크 분석·문서 자동화
핵심 스택	Python, FastAPI, LangGraph, ChromaDB, GPT-4o, Docker, GCP Cloud Run
출발점	n8n 로우코드 프로토타입 → Python 서비스로 재설계

문제

구매 담당자는 수백 건의 SKU 단위 재고 데이터를 검토하고, 이를 토대로 분석 보고서·구매 요청서·공급업체 이메일 같은 문서를 작성하는 데 상당한 수작업을 들입니다.

문제는 이 과정이 **사람의 상황에 크게 좌우된다**는 점입니다. 바쁜 일정이나 담당자의 숙련도 등의 이슈로 인해 **공급업체의 납기·품질 이력, 과거 협상 조건, 통관·규제·안전 이슈, 품목의 시장 반응**처럼 여러 문서에 흩어진 근거 자료를 미처 다 확인하지 못하면 — 그 결과는 곧 비효율적인 발주 및 잠재적 이윤 손실로 이어집니다.

무엇을 하는가

재고 스냅샷을 입력받으면 시스템이 네 가지 결과물을 자동으로 생성합니다.

- 구매 분석 보고서** — 공급업체별로 그룹화된 재고 리스크 평가 및 총당 계획 수립
- 품질 평가 보고서** — 별도 에이전트가 분석 결과의 데이터·논리 일관성을 감사
- 구매 요청서(Purchasing Request, PR)** — 승인 절차용으로 공급업체별 서식화
- 공급업체 이메일 초안** — 납기·재고 가용성 문의용 외부 커뮤니케이션

재고 스냅샷은 재고 부족이 우려되는 품목을 추린 CSV로 입력되고, 공급업체·아이템 이력 문서는 업로드해 벡터 DB에 임베딩한 뒤 분석 단계에서 자동으로 검색됩니다.

아키텍처

PoC 기반 전환(PoC-to-Production) — n8n 프로토타입에서 프로덕션 코드로

처음에는 **n8n** 로우코드 워크플로우로 개념을 빠르게 검증했습니다. 노드를 연결해 “CSV → 그룹화 → LLM 분석 → 문서 생성”의 흐름이 실제로 동작하는지 확인하는 단계였습니다.

검증이 끝난 뒤, 복잡한 분기 로직·상태 관리·컨테이너 배포를 구현하기 위해 전체를 **Python(FastAPI + LangGraph)**으로 재설계했습니다. 이 과정에서 노드 기반 흐름을 비동기 API와 상태 그래프로 옮기고, API 키 인증·스트리밍을 갖춘 배포 가능한 서비스로 다듬었으며, 공개 데모는 CI/CD(GitHub Actions → Cloud Run)까지 구성했습니다 (실제 운영은 회사 로컬 서버 구동).

멀티 에이전트 파이프라인 (LangGraph)

5개의 특화된 에이전트를 LangGraph 상태 그래프로 연결하고, 검증 노드에서 필요 시 이전 단계로 되돌아가 재작성합니다.



왜 멀티 에이전트인가

하나의 거대한 프롬프트로 전체를 처리하는 방식에 비해 다음과 같은 장점이 있습니다.

- **모듈화(Modularity)** — 각 단계를 서로 건드리지 않고 개별적으로 교체·개선할 수 있어, 실제로 분석 에이전트만 떼어 GPT-4o에서 자체 호스팅 Llama로 대체 가능성을 검증했습니다.
- **조기 오류 차단** — 평가 에이전트가 분석 결과를 독립적으로 감사하기 때문에, 문제가 있는 분석이 뒤 단계 (PR·이메일)로 그대로 전파되기 전에 걸러집니다.
- **정밀한 자기 교정 및 검증** — 검증 노드가 문제를 감지하면 전체 파이프라인을 처음부터 다시 돌릴 필요 없이, 문제가 생긴 단계만 정확히 지목해 재작성을 지시할 수 있습니다.

핵심 구성 요소

RAG 지식 베이스 (ChromaDB)

- **파이프라인** — 공급업체 기록·아이템 이력 PDF(개별 또는 ZIP 대량 업로드)를 청킹(1,000자, 200자 오버랩)해 OpenAI 임베딩으로 변환하고, 문서 유형별(공급업체 이력·품목 이력·보고서/PR/이메일 예시)로 분리된 5개의 독립 컬렉션에 저장합니다.
- **실증적 하이퍼파라미터 튜닝(Hyperparameter Tuning)** — **청킹 사이즈 결정** — 실제 공급업체·품목 이력 문서를 검토해보니 보통 A4 1~2페이지 분량으로 작성되는 걸 확인했고, 이를 기준으로 청크 크기를 500자·1,000자·1,500자로 나눠 저장·검색 효율을 비교 테스트했습니다. 청크가 너무 작으면(500자) 하나의 사건(원인 → 결과) 서술이 청크 경계에서 잘려 문맥이 끊기고, 너무 크면(1,500자) 서로 다른 사건 여러 개가 한 청크에 뒤섞여 임베딩이 특정 사건 하나에 집중되지 못하고 흐려지는 문제가 있었습니다. 실제 문서의 “사건 단위” 서술 길이와 가장 잘 맞아떨어진 1,000자를 최종 청크 크기로 정했고, 오버랩은 청크 크기의 20% 수준인 200자로 설정해 사건 서술이 경계에 걸쳐 있어도 앞뒤 청크 모두에서 놓치지 않도록 했습니다.
- **현업 워크플로우 맞춤형 설계** — 실제 공급업체·품목 이력 문서는 서두에 공급업체명·아이템 코드를 명시하는 원칙을 따르고 있었습니다. 이를 파악한 뒤, 별도의 복잡한 온톨로지 구현 없이 **정규식만으로 신뢰할 수 있는 메타데이터**를 뽑아낼 수 있었습니다.
 - **메타데이터 태깅** — 인제스트 시점에 정규식으로 공급업체명·아이템 코드를 추출해 문서에 메타데이터로 부여합니다. 청킹 이후에도 모든 하위 청크가 원본 문서의 메타데이터를 그대로 상속받으므로, 특정 청크의 텍스트에 그 이름이 등장하지 않아도 필터는 항상 정확하게 걸립니다.
 - **명시적 메타데이터 필터링 + 유사도 검색 (2단계 검색)** — 분석 에이전트가 히스토리를 조회할 때, LLM이 생성한 쿼리 텍스트에만 의존하지 않고 파이프라인이 이미 알고 있는 실제 공급업체명·아이템 코드를 메타데이터 필터로 함께 전달합니다. 1차로 이 필터를 만족하는 문서로 후보군을 좁히고, 2차로 그 안에서 유사도 순위를 매겨 상위 K개를 가져오는 방식이라, 결과가 “의미적으로 비슷한 문서”가 아니라 “그 공급업체·품목에 실제로 해당하는 문서 중 의미적으로 가장 관련 있는 것”으로 정확히 좁혀집니다.
 - **강점** — 순수 시맨틱 유사도 검색만 쓰면 다른 공급업체의 유사한 서술(예: 비슷한 배송 지연 사례)이 섞여 들어올 위험이 있는데, 메타데이터 필터가 이런 교차 오염을 원천 차단합니다. 그것도 온톨로지·지식 그래프 같은 무거운 구조 없이, 이미 존재하던 문서 작성 관행 하나로 달성한 것입니다.
- **목적에 따른 차별화된 검색 전략 활용** — 공급업체·품목 이력은 “사실 검색”이라 메타데이터 필터 + 유사도 랭킹을 쓰지만, 분석 보고서·PR·이메일의 모범 양식 예시는 “스타일 검색”이라 성격이 다릅니다. 회사 서식이 바뀌면 예전 예시가 최신 문서와 스타일이 달라질 수 있는데, 유사도만으로는 이 차이를 구분하지 못합니다. 그래서 예시 컬렉션은 인제스트 시점에 타임스탬프를 남기고, 검색 시 유사도가 아니라 **가장 최근에 업로드된 순서**로 상위 N개를 가져오도록 설계했습니다.

AI 품질 평가(Evaluator) 에이전트

별도 평가 에이전트가 분석 에이전트의 출력(JSON)과 그 근거가 된 공급업체·품목 이력 원문을 대조해, **데이터 정확성·이력 충실도(할루시네이션 여부)·논리적 추론·운영 적합성** 4개 기준으로 10점 만점 채점합니다. 재조회가 아니라 분석 에이전트가 실제로 조회했던 이력 원문을 그대로 전달받아 대조하기 때문에, 실제로는 근거가 있는 서술을 “출처 불명”으로 오판해 허위 할루시네이션으로 잘못 채점하는 문제를 방지합니다.

가드레일(Guardrail) — 자기 교정 루프

이메일 결과물은 검증 노드를 거치며 내부 정보 유출 여부를 점검합니다. 유출이 감지되면 그대로 반환하지 않고 그래프가 초안 재생성으로 되돌아갑니다.

실시간 스트리밍 (SSE)

단일 요청-응답 대신, 파이프라인 각 단계(CSV 파싱 → 그룹화 → 분석 → 문서 생성)의 진행 상황을 Server-Sent Events로 클라이언트에 실시간 전송함으로써 사용자에게 “지금 어느 단계인지”를 실시간으로 보여주고, 오류 발생 시에도 즉시 알릴 수 있습니다.

하이브리드(Hybrid) 설계 — 결정론적 계산과 LLM 추론의 역할 분리

공급업체 그룹화 단계에서 `WksTo005` (품질까지 남은 주 수)와 `CurrentStock` 을 바탕으로 권장 발주일과 수량을 파이썬 산식으로 계산합니다. 날짜·수량처럼 정확도가 중요한 값은 LLM 추론에 맡기지 않고 결정론적 코드로 처리해 할루시네이션 리스크를 없앴고, LLM은 그 결과를 받아 우선순위가 정리된 구조화 데이터를 대상으로 리스크 해석과 문서 작성에만 집중합니다.

보안(Security) & 관측성(Observability)

보안: 공개 데모는 익명 사용자의 토큰 남용을 막기 위해 헤더 기반 API 키 인증(`X-API-Key`)과 IP 단위 요청 제한을 두었습니다. 사내 배포는 내부망 ID/PW 개인별 로그인을 구현했습니다. 관측성: LangSmith로 모든 노드의 토큰 사용량·지연·프롬프트 성능을 추적하고, LangChain 밖의 자체 로직도 `@traceable` 로 계측합니다.

배포

공개 데모는 Docker 컨테이너로 GCP Cloud Run(서버리스, 스케일 투 제로)에 올려 누구나 접속할 수 있게 했고, CI/CD는 GitHub Actions로 구성해 `main` 브랜치에 푸시할 때마다 이미지를 빌드·자동 배포합니다. 실제 사내 운영에서는 SAP ERP가 상시 구동되는 24시간 로컬 서버에 앱을 상주시켜 내부망에서 팀이 사용하도록 구성했으며, 이 환경에서는 콜드 스타트나 벡터스토어 휘발성 문제가 없습니다.

파인튜닝 연구: Llama-3-8B에 SFT + DPO 적용

GPT-4o 분석 에이전트를 자체 호스팅 모델로 대체할 수 있는지를 테스트했습니다. 목적은 호출당 비용과 데이터 프라이버시 노출을 줄이는 것이고, 접근 방식은 GPT-4o의 지식을 Llama-3-8B로 증류(distillation)하는 것이었습니다.

학습 데이터

- **데이터 출처** — 실제 회사 데이터 대신 GPT-4o로 생성한 **합성 구매 시나리오**를 사용했습니다.
- **시나리오 구성** — 재고 행(품목코드·품목명·공급업체·리스크 등급·현재고·품질까지 주수)과 공급업체·품목 이력(자연어)으로 구성됩니다.
- **다양성** — 배송 지연·가격 급등·품질 이슈·수요 변동 같은 이력 패턴을 다양하게 담았습니다.
- **적대적(adversarial) 케이스 포함** — 데이터 모순(재고는 많은데 품질 임박)·상충 맥락(신뢰 표기 vs 최근 실패)·모호한 이력·결측값 같은 케이스도 의도적으로 포함해 강건성을 높였습니다. 이런 케이스를 학습한 모델은 실전에서 유사한 상황을 마주쳐도, 모순을 얼버무리거나 없는 정보를 지어내는 대신 **모순·결측을 있는 그대로 명시**하도록 반응합니다.

- **지식 증류(Knowledge Distillation)** — 이 시나리오에 대해 GPT-4o가 만든 분석 결과(JSON: 분석 보고서·핵심 질문·보충 타임라인)를 정답으로 삼아 Llama가 이를 재현하도록 학습했습니다.
- **파인튜닝 범위** — 파이프라인 전체가 아니라 대체하려는 **분석 에이전트의 출력**으로 한정했습니다.

1단계 — 지도 파인튜닝 (SFT)

- Llama-3-8B + QLoRA (4-bit, LoRA rank 16 — 학습 파라미터 약 41M / 8B, 0.51%)
- 교사 예제 30개 (GPT-4o가 시나리오별로 생성한 분석 JSON)
- **학습 데이터 형태**: JSONL, `{instruction, input: {inventory, supplier_history, item_history}, output: {analysis: {...}}}` — 정답 라벨은 `output.analysis` (분석 보고서·핵심 질문·보충 타임라인)만 사용
- Vertex AI, Tesla T4 1장, 5 epoch, 367초
- 학습 손실: 1.14 → 0.41

2단계 — 직접 선호 최적화 (DPO)

- 선호 쌍 25개 (홀드아웃 5개 제외; chosen: GPT-4o 출력, rejected: SFT 출력)
- **학습 데이터(선호 쌍) 형태**: JSONL, `{prompt, chosen, rejected}` — `prompt` 는 SFT와 동일한 템플릿 (Response 제외), `chosen` 은 GPT-4o 정답 analysis JSON, `rejected` 는 동일 프롬프트에 대해 SFT 모델이 실제 생성한 출력
- Vertex AI, Tesla T4, 3 epoch, 약 14분
- Unsloth `PatchDPOTrainer()` + TRL `DPOTrainer`

평가 (GPT-4o-as-judge, 홀드아웃 5개)

모델	평균 점수	JSON 유효율	채점 기준
Base Llama-3-8B	0.0 / 10	0%	GPT-4o가 다음 2개 기준을 각 10점 만점으로 채점한 뒤 평균: (1) 데이터 정확성 — 공급업체명·아이템 코드·재고·리스크 등급 등 입력값을 정확히 반영했는가, (2) 추론 품질 — 재고 총량 분석과 핵심 질문이 논리적으로 타당한가. (프로덕션 AI 품질 평가 에이전트의 4개 기준과는 별개의, 이 벤치마크 전용 간이 채점 방식입니다.)
Llama-3-8B SFT	9.3 / 10	100%	
Llama-3-8B SFT + DPO	7.4 / 10	100%	
GPT-4o (기준)	10.0 / 10	100%	

결과 해석

- **SFT 성과** — GPT-4o 대비 약 93%의 평가 품질에 도달했고, 모든 예제에서 유효한 구조화 출력을 냈습니다. 자체 호스팅 모델로의 대체 가능성을 뒷받침하는 결과입니다.
- **DPO 성능 하락** — SFT 대비 오히려 하락(9.3 → 7.4)했습니다. 선호 쌍이 25개(홀드아웃 제외 시 20개)로 적었던 영향도 있습니다.

- **원인: 선호 쌍 구성 방식** — chosen/rejected를 실제 품질 평가 없이 “GPT-4o 출력 vs SFT 자체 출력”으로 고정 배정했습니다. 개별 사례마다 SFT 출력이 실제로 더 나쁜지 검증되지 않았고, 모델이 내용의 논리적 깊이보다 GPT-4o의 문체를 모방하는 방향으로 편향됐을 가능성이 있습니다.
- **관찰과의 정합성** — 데이터 정확도(8~9/10)는 유지된 채 추론 깊이만 알려진 결과가 이 가설과 일치합니다.
- **향후 개선 방향** — SFT 모델이 생성한 여러 후보를 GPT-4o judge로 랭킹해, 실제 품질 차이가 검증된 on-policy 선호 쌍을 구성하는 방식을 시도할 필요가 있습니다.

성과

- **자동화 및 분석 품질 향상** — 반복적인 SKU 단위 재고 검토와 문서 작성을 자동화 — 담당자·상황에 따라 들쭉날쭉하던 판단을 일관된 품질로 끌어올림.
- **RAG 기반 컨텍스트 분석** — 사람이 하면 담당자의 분석 속도·숙련도에 따라 누락되기 쉬웠던 공급업체 이력·리드 타임 등 다중 소스 컨텍스트를, RAG로 몇 초 만에 모두 끌어와 결합함으로써 리스크 우선순위 기반 의사결정을 빠르고 일관되게 지원.
- **자체 호스팅 대체 가능성 검증** — 파인튜닝한 로컬 모델(SFT)이 GPT-4o 대비 약 93% 품질에 도달함을 실증. 자체 호스팅 전환 시 API 호출 비용 제로화와 데이터 외부 미전송(프라이버시 확보)이라는 두 가지 실익 확인.
- **운영 지향 설계** — SSE 실시간 스트리밍(대기 중에도 진행 상황 실시간 확인, 타임아웃 리스크 회피), 자기 교정 에이전트 그래프(민감정보 유출 자동 검출·재작성), LangSmith 기반 관측성(노드별 토큰·지연·프롬프트 성능 및 비용 모니터링과 문제 원인 추적)을 갖춘 설계.
- **정확성과 신뢰성 확보** — 정확도가 중요한 계산(발주일·수량)은 결정론적 산식으로, 맥락 판단이 필요한 부분은 LLM 추론으로 역할을 분리.
- **현업 맞춤형 효율화 설계** — 실제 문서 작성 관행(공급업체명·아이템 코드를 서두에 명시)을 파악한 뒤, 온톨로지 같은 무거운 구조화 없이 정규식 메타데이터 추출 + 강제 필터링만으로 RAG 검색의 교차 오염(다른 공급업체·품목 문서가 섞여 들어오는 문제)을 해결. 일반적인 엔지니어링 패턴을 그대로 적용하기보다, 현업의 실제 작업 방식을 관찰해 거기 맞춘 가볍고 효율적인 해법을 설계.

향후 과제

GPT-4o → 자체 호스팅 모델 전환

현재 분석 에이전트는 GPT-4o 호출에 의존하므로 호출 비용이 발생하고 데이터가 외부로 전송됩니다. 이를 줄이기 위해 자체 호스팅 모델을 검증했고, SFT만으로 GPT-4o 대비 약 93% 품질에 도달해 대체 타당성을 확인했습니다(Llama-3-8B 기준). 다만 실제 전환까지는 다음 과제들이 남아 있습니다.

- **DPO 훈련셋 설계 정교화** — 현재는 chosen/rejected를 품질 평가 없이 “GPT-4o 출력 vs SFT 자체 출력”으로 고정 배정해, 모델이 내용의 논리적 깊이보다 GPT-4o의 문체를 모방하는 방향으로 편향됐을 가능성이

있습니다. chosen을 GPT-4o 출력으로 유지하는 건 증류(distillation)의 목표(모델 자신의 실력을 넘어서는 외부 기준점 제공)상 타당하지만, rejected는 SFT 모델의 여러 후보 중 judge가 “내용·논리 품질이 실제로 더 낫다”고 확인한 것만 선별하거나, judge에게 문체가 아닌 내용만 평가하도록 기준을 명시해 스타일 차이가 선호 신호에 섞이지 않도록 개선할 필요가 있습니다.

- **선호 쌍 규모 확대** — 현재 25개(홀드아웃 제외 시 20개)뿐인 선호 쌍을 더 확보해 DPO 학습 신호를 안정화합니다.
- **최신 모델 활용 검토** — GPT-4o는 이 프로젝트에서 프로덕션 분석 에이전트(반복 호출, 비용 민감)이자 학습 데이터를 만드는 교사(일회성 호출, 비용 무관)라는 두 가지 역할을 겸합니다. 교사 쪽은 GPT-5 같은 상위 모델로 바뀌도 일회성 비용이라 부담이 적어 학습 데이터 품질을 높이는 데 활용할 수 있고, 학생(대체) 쪽은 Llama-3-8B 외에 이후 공개된 Qwen, Llama 최신 버전 등 오픈소스 소형 모델을 비교 검증해 프로덕션 비용·성능을 함께 개선할 여지가 있습니다.

RAG 검색 확장 아이디어 — 온톨로지/지식 그래프 적용 검토

현재 방식(메타데이터 필터 + 시맨틱 유사도)은 공급업체·품목 단위로 관련 문서를 빠르고 정확하게 찾아온다는 본래 목적에는 충분합니다. 여기서 한 걸음 더 나아가, “이 공급업체가 납품하는 다른 품목 중 과거 유사 품질 이슈가 있었던 것은?”처럼 여러 문서를 넘나드는 관계·집계 질의까지 지원하고 싶다면, 온톨로지/지식 그래프를 추가로 검토해볼 수 있습니다.

- **기술 접근 — 지식 그래프(Knowledge Graph) 결합** — “공급업체 A는 품목 X를 납품한다”, “품목 X는 작년에 품질 이슈가 있었다”처럼 데이터 간의 **관계 자체**를 지도처럼 그려두는 경량 지식 그래프(예: Neo4j)를 벡터 검색과 함께 씁니다. 지금처럼 “의미가 비슷한 문서 찾기”는 벡터 검색이 맡고, “이 공급업체와 연결된 품목·이슈를 따라가며 찾기”는 지식 그래프가 맡는 식으로 역할을 나눕니다.
 - **기대 효과 — 다중 홉 추론 및 집계 질의(Multi-hop Reasoning & Aggregation)** — 지금은 한 문서 안에서 답을 찾는 질문(이 공급업체 이력이 어떻다)에 강한데, 여러 문서·엔티티를 이어가야 답이 나오는 질문(“이 공급업체가 납품하는 다른 품목 중에도 문제 있었던 게 있나?”)이나, 셀 수 있는 질문(“이번 분기 자연이 2번 이상인 공급업체는 몇 곳?”)에도 정확하게 답할 수 있게 됩니다.
 - **도입 조건 — 단계적 적용** — 지식 그래프를 만들려면 관계를 미리 정의하고 데이터에서 뽑아내는 작업이 추가로 필요합니다. 그러니 실제 사용 중에 이런 “여러 문서를 넘나드는 질문”이 자주 나온다는 게 확인된 다음에, 필요한 만큼만 점진적으로 붙이는 게 현실적입니다.
-

기술 스택

영역	기술
백엔드	Python, FastAPI (async, SSE)
오케스트레이션	LangGraph (상태 기반 멀티 에이전트), LangChain
LLM	GPT-4o / GPT-4o-mini; Llama-3-8B (QLoRA)
파인튜닝	Unsloth, TRL (SFTTrainer / DPOTrainer), Vertex AI, W&B
벡터 DB	ChromaDB (RAG)
관측성	LangSmith
프론트엔드	React, TypeScript, Framer 커스텀 코드 컴포넌트
데이터 처리	Pandas, PyPDF, python-docx
인프라 / CI-CD	Docker, GCP Cloud Run, Vertex AI, Artifact Registry, GitHub Actions